

OpenAI Prompt Caching — Code Documentation

Full walkthrough of `openai_prompt_caching.py`, explaining every section in detail.

Setup requirements

Before running the code:

```
bash

pip install openai python-dotenv
```

Create a file named `.env` in the same directory as the script:

```
OPENAI_API_KEY=sk-proj-your-real-key-here
```

Important: Never commit `.env` to git. Add it to `.gitignore`.

Section 1: The module docstring

```
python

"""
Basic prompt caching example with the OpenAI API.
...
"""
```

A multi-line string at the very top of a Python file becomes the **module docstring** — accessible via `help(module_name)` or `module.__doc__`. It's not just a comment; it's documentation that IDEs, `pydoc`, and other tools can read programmatically.

The docstring includes:

- What the script demonstrates
 - Key OpenAI caching facts (1024-token minimum, 128-token increments, ~50% discount)
 - Setup instructions
-

Section 2: The imports

```
python
```

```
import os
import time
from dotenv import load_dotenv
from openai import OpenAI
```

Each import serves a specific purpose:

Import	Purpose
<code>os</code>	Provides <code>os.getenv()</code> to read environment variables — how we keep the API key out of source code
<code>time</code>	Provides <code>time.perf_counter()</code> , the most accurate timer for measuring short durations
<code>load_dotenv</code>	Reads a <code>.env</code> file from disk and copies its contents into <code>os.environ</code>
<code>OpenAI</code>	The official OpenAI client class

Without `load_dotenv`, your `.env` file would just be a text file Python ignores entirely.

Section 3: Loading the API key from .env

```
python

load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")
if not api_key:
    raise RuntimeError(
        "OPENAI_API_KEY not found. Create a .env file with: OPENAI_API_KEY=sk-..."
    )

client = OpenAI(api_key=api_key)
```

Step-by-step

`load_dotenv()` searches for a file named `.env` starting in the current directory (and walks up parent directories if needed). When found, it parses lines like `OPENAI_API_KEY=sk-proj-abc123` and injects them into `os.environ`. After this call, the variables behave as if you'd set them in your shell with `export` before running the script.

`os.getenv("OPENAI_API_KEY")` reads the variable. Returns the string value if found, or `None` if missing.

Why not `os.environ["OPENAI_API_KEY"]`? That syntax raises `KeyError` if the variable is missing — `os.getenv()` returns `None` instead, which is easier to check.

The `if not api_key:` check catches three failure modes in one line:

1. The `.env` file doesn't exist
2. The variable name is misspelled
3. The value is an empty string

Failing fast here with a clear message saves you from a confusing 401 Unauthorized error 30 lines later.

`OpenAI(api_key=api_key)` creates the client. The SDK would auto-discover `OPENAI_API_KEY` from the environment if you wrote `OpenAI()` with no arguments — but passing it explicitly makes the data flow visible.

Why use `.env` at all?

Benefit	Explanation
Security	Keeps secrets out of git (add <code>.env</code> to <code>.gitignore</code>)
Flexibility	Different keys for dev/staging/prod without code changes
Documentation	Share <code>.env.example</code> to show required config without leaking values

Section 4: The static prefix

```
python

LARGE_SYSTEM_PROMPT = """
You are a customer support assistant for ACME Robotics.
...
""" * 30
```

Three things matter here for caching to work:

1. The content is constant

Every byte is identical across all three calls in the script. Caching requires **byte-for-byte identical** prefix matching.

2. It's defined at module scope

Defining it inside `ask()` would risk subtle differences between calls (timestamps, random ordering, whitespace from string formatting). Module-scope definition guarantees identity.

In Python, `"string" * 30` repeats the string 30 times. This is a quick hack to push past OpenAI's **1024-token minimum**.

Below that threshold, OpenAI silently ignores caching — you'd see `cached_tokens: 0` forever with no error message. In production you'd remove this multiplier because your real system prompt + RAG context + tool definitions would already be long enough.

Section 5: The `ask` function

```
python

def ask(question: str, label: str):
    start = time.perf_counter()

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": LARGE_SYSTEM_PROMPT},
            {"role": "user", "content": question},
        ],
        max_tokens=200,
    )

    elapsed = time.perf_counter() - start
    usage = response.usage
    ...
```

Component breakdown

`time.perf_counter()` captures a high-resolution timestamp. It's monotonic (never goes backwards if the system clock changes) and has sub-microsecond resolution — preferred over `time.time()` for measuring durations.

`client.chat.completions.create(...)` is the actual HTTP request to `/v1/chat/completions`. The SDK handles authentication headers, JSON serialization, retries on transient errors, and response parsing.

`model="gpt-4o-mini"` picks the model. Any modern chat model supports caching; smaller ones are cheaper for demos.

The critical part: message ordering

OpenAI caches the **prefix** — bytes from position 0 forward, stopping at the first byte that differs from a previous request. So:

```
python

messages=[
    {"role": "system", "content": LARGE_SYSTEM_PROMPT},    # static - caches
    {"role": "user", "content": question},                # dynamic - doesn't
]
```

If you **reversed** these — user message first, system second — the question would be at the start of the prompt. Since the question changes every call, the entire prompt would mismatch and **nothing would cache**.

└ **Cardinal rule of caching: static content first, dynamic content last.**

`max_tokens=200` caps response length. Output tokens are billed at full price regardless of caching, so capping output keeps demo costs predictable.

Section 6: Extracting the cache metrics (deep dive)

```
python

cached = 0
details = getattr(usage, "prompt_tokens_details", None)
if details is not None:
    cached = getattr(details, "cached_tokens", 0) or 0
```

This is the trickiest part of the code. Let me explain why it's structured this way.

What we're trying to do

Answer one question: **how many input tokens were served from cache?** That's the whole point of the demo.

The response object structure

The cached-token count is nested **two levels deep**:

```
response
└─ usage
    ├── prompt_tokens      (total input tokens)
    ├── completion_tokens  (output tokens)
    └─ prompt_tokens_details
        └─ cached_tokens   ← the value we want
```

The naive approach would be:

```
python
```

```
cached = response.usage.prompt_tokens_details.cached_tokens
```

This works most of the time. But "most of the time" isn't good enough.

Three things that can go wrong

Problem 1: `prompt_tokens_details` might not exist. OpenAI added this field in late 2024 specifically for cache metrics. Older SDK versions don't have it. Direct access would raise:

```
AttributeError: 'CompletionUsage' object has no attribute 'prompt_tokens_details'
```

Problem 2: `cached_tokens` might be `None`. On cold-start requests, the API sometimes returns `cached_tokens=None` instead of `0`. Using that value would break downstream:

```
python

print(f"cached: {cached}")          # prints "cached: None"
print(f"savings: {cached * 0.5}")  # TypeError: NoneType * float
```

Problem 3: Attribute exists but value is `None` vs attribute doesn't exist. These are different cases in Python. We don't want to care which one we're in.

How the code handles all three

Line 1: `cached = 0`

Assume the worst. If everything below fails, `cached` stays at `0` and the program continues. This is the "safe default" pattern.

Line 2: `details = getattr(usage, "prompt_tokens_details", None)`

The `getattr()` built-in takes three arguments:

```
getattr(object, attribute_name, default_value)
```

It says: "Look on `object` for an attribute named `attribute_name`. If found, return its value. If not, return `default_value` instead of raising `AttributeError`."

After this line, `details` is either:

- A `PromptTokensDetails` object (modern SDK)
- `None` (old SDK, or field absent)

Line 3: `if details is not None:`

We only proceed if we got something back.

Why `is not None` instead of `if details:`? Some objects are "falsy" even when they exist (empty list, 0, empty string). Using `is not None` precisely checks "did I get anything back at all?"

Line 4: `cached = getattr(details, "cached_tokens", 0) or 0`

Two safety checks combined.

Part A — `getattr(details, "cached_tokens", 0)`: Get the attribute, or fall back to 0 if missing. Possible results:

- Integer like `1408` (happy path)
- `None` (attribute exists but value is null)
- `0` (default kicked in because attribute missing)

Part B — `or 0`: In Python, `A or B` returns `A` if `A` is truthy, otherwise `B`.

Expression	Result	Why
<code>1408 or 0</code>	<code>1408</code>	1408 is truthy
<code>None or 0</code>	<code>0</code>	None is falsy
<code>0 or 0</code>	<code>0</code>	0 is falsy

This collapses the `None` case into `0`. By the end, `cached` is **guaranteed** to be an integer.

Why bother with all this defensive code?

For a script you're running yourself, you could write the one-liner version. But this code is a **teaching example** that other people will run, possibly with different SDK versions. A crash on a tutorial script with a cryptic `AttributeError` is a terrible first impression. The defensive code guarantees the demo always prints something sensible.

Mental model: whenever you're reading a field from an external API response, assume any layer of the nesting could be missing or null. `getattr()` with a default is your friend.

Section 7: The output formatting

```
python
```

```

print(f"\n--- {label} ---")
print(f"Question      : {question}")
print(f"Elapsed        : {elapsed:.2f}s")
print(f"prompt_tokens   : {usage.prompt_tokens}")
print(f"  cached_tokens  : {cached}  <-- this is the cache hit count")
print(f"completion_tokens: {usage.completion_tokens}")
print(f"Answer          : {response.choices[0].message.content[:160]}...")

```

What's happening

f-strings (the `f"..."` syntax) let you embed Python expressions inside `{}`.

`{elapsed:.2f}` formats `elapsed` as a float with 2 decimal places.

Visual padding (`(Question :)`) lines up the colons for scannable output.

`response.choices[0].message.content` gets the actual text reply. The `choices` list exists because the API *could* return multiple completions in one call (via the `n` parameter), but we only ask for one — so it's always `[0]`.

`[:160]` slices to the first 160 characters. Full answers would make demo output noisy.

Section 8: The `if __name__ == "__main__":` block

```

python

if __name__ == "__main__":
    ask("What's your cheapest robot?", label="First call (cache MISS)")
    ask("Does the ACME-300 have LiDAR?", label="Second call (cache HIT)")
    ask("What's the return policy on opened items?", label="Third call (cache HIT)")

```

The guard explained

Every Python file has a special variable `__name__`:

- When you run the file directly (`(python openai_prompt_caching.py)`), `__name__ == "__main__"`
- When the file is imported (`(import openai_prompt_caching)`), `__name__ == "openai_prompt_caching"`

The guard means: **"only execute this block when run as a script, not when imported as a library."** It's good hygiene — without it, importing the file would fire three API calls as a side effect.

Why three calls?

Two would leave ambiguity ("maybe the second call was a fluke"). Three establishes the pattern — you see the miss, then two consecutive hits.

Why these questions specifically?

Each question is 15–25 tokens. The system prompt is thousands of tokens. So roughly 99% of the input is identical between calls — the ideal scenario for caching.

What you'll see when you run it

First call (cold start)

```
prompt_tokens    : 1543
  cached_tokens  : 0          ← cold start, cache being written
Elapsed          : 2.15s
```

Second and third calls (warm cache)

```
prompt_tokens    : 1543
  cached_tokens  : 1408      ← almost the entire system prompt from cache
Elapsed          : 1.12s    ← noticeably faster
```

Key observation

Notice `prompt_tokens` stays the same — caching doesn't **reduce** your token count, it makes a **portion** of those tokens cheaper. You're still billed for 1,543 input tokens, but 1,408 of them are at the discounted cached rate.

Common debugging scenarios

Symptom	Likely cause	Fix
<code>cached_tokens: 0</code> on every call	Prompt under 1,024 tokens	Make <code>LARGE_SYSTEM_PROMPT</code> longer
<code>cached_tokens: 0</code> after long pause	TTL expired (5–10 min)	Send another call within the window
<code>AttributeError: 'CompletionUsage'</code>	Old <code>openai</code> package	<code>pip install --upgrade openai</code>

Symptom	Likely cause	Fix
<code>RuntimeError: OPENAI_API_KEY not found</code>	<code>.env</code> missing/misnamed	Create <code>.env</code> in script directory
401 Unauthorized	Invalid or expired key	Check <code>.env</code> content, regenerate key
Cache hits inconsistent	OpenAI cache is best-effort	Normal — some misses under load

Key takeaways

1. Use `.env` to keep API keys out of source code
2. **Static content first, dynamic content last** in your messages list
3. **Prompts must be $\geq 1,024$ tokens** for OpenAI caching to activate
4. Read `cached_tokens` from `prompt_tokens_details`, not the top level of `usage`
5. Use `getattr()` with defaults when reading optional API response fields
6. Use `if __name__ == "__main__":` to make scripts safe to import